

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §3.6.

4.7 Adaptive Quadrature

The idea behind adaptive quadrature is very simple. Suppose you have two different numerical estimates I_1 and I_2 of the integral

$$I = \int_a^b f(x) dx \quad (4.7.1)$$

Suppose I_1 is more accurate. Use the relative difference between I_1 and I_2 as an error estimate. If it is less than ϵ , accept I_1 as the answer. Otherwise divide the interval $[a, b]$ into two subintervals,

$$I = \int_a^m f(x) dx + \int_m^b f(x) dx \quad m = (a + b)/2 \quad (4.7.2)$$

and compute the two integrals independently. For each one, compute an I_1 and I_2 , estimate the error, and continue subdividing if necessary. Dividing any given subinterval stops when its contribution to ϵ is sufficiently small. (Obviously recursion will be a good way to implement this algorithm.)

The most important criterion for an adaptive quadrature routine is reliability: If you request an accuracy of 10^{-6} , you would like to be sure that the answer is at least that good. From a theoretical point of view, however, it is impossible to design an adaptive quadrature routine that will work for all possible functions. The reason is simple: A quadrature is based on the value of the integrand $f(x)$ at a *finite* set of points. You can alter the function at all the other points in an arbitrary way without affecting the estimate your algorithm returns, while the true value of the integral changes unpredictably. Despite this point of principle, however, in practice good routines are reliable for a high fraction of functions they encounter. Our favorite routine is one proposed by Gander and Gautschi [1], which we now describe. It is relatively simple, yet scores well on reliability and efficiency.

A key component of a good adaptive algorithm is the termination criterion. The usual criterion

$$|I_1 - I_2| < \epsilon |I_1| \quad (4.7.3)$$

is problematic. In the neighborhood of a singularity, I_1 and I_2 might never agree to the requested tolerance, even if it's not particularly small. Instead, you need to somehow come up with an estimate of the *whole* integral I of equation (4.7.1). Then you can terminate when the error in I_1 is negligible compared to the whole integral:

$$|I_1 - I_2| < \epsilon |I_s| \quad (4.7.4)$$

where I_s is the estimate of I . Gander and Gautschi implement this test by writing

```
if (is + (i1-i2) == is)
```

which is equivalent to setting ϵ to the machine precision. However, modern optimizing compilers have become too good at recognizing that this is algebraically equivalent to

```
if (i1-i2 == 0.0)
```

which might never be satisfied in floating point arithmetic. Accordingly, we implement the test with an explicit ϵ .

The other problem you need to take care of is when an interval gets subdivided so small that it contains no interior machine-representable point. You then need to terminate the recursion and alert the user that the full accuracy might not have been attained. In the case where the points in an interval are supposed to be $\{a, m = (a + b)/2, b\}$, you can test for $m \leq a$ or $b \leq m$.

The lowest order integration method in the Gander-Gautschi method is the four-point Gauss-Lobatto quadrature (cf. §4.6)

$$\int_{-1}^1 f(x) dx = \frac{1}{6} \left[f(-1) + f(1) \right] + \frac{5}{6} \left[f\left(-\frac{1}{\sqrt{5}}\right) + f\left(\frac{1}{\sqrt{5}}\right) \right] \quad (4.7.5)$$

This formula, which is exact for polynomials of degree 5, is used to compute I_2 . To reuse these function evaluations in computing I_1 , they find the seven-point Kronrod extension,

$$\begin{aligned} \int_{-1}^1 f(x) dx = & \frac{11}{210} \left[f(-1) + f(1) \right] + \frac{72}{245} \left[f\left(-\sqrt{\frac{2}{3}}\right) + f\left(\sqrt{\frac{2}{3}}\right) \right] \\ & + \frac{125}{294} \left[f\left(-\frac{1}{\sqrt{5}}\right) + f\left(\frac{1}{\sqrt{5}}\right) \right] + \frac{16}{35} f(0) \end{aligned} \quad (4.7.6)$$

whose degree of exactness is nine. The formulas (4.7.5) and (4.7.6) get scaled from $[-1, 1]$ to an arbitrary subinterval $[a, b]$.

For I_s , Gander and Gautschi find a 13-point Kronrod extension of equation (4.7.6), which lets them reuse the previous function evaluations. The formula is coded into the routine below. You can think of this initial 13-point evaluation as a kind of Monte Carlo sampling to get an idea of the order of magnitude of the integral. But if the integrand is smooth, this initial evaluation will itself be quite accurate already. The routine below takes advantage of this.

Note that to reuse the four function evaluations in (4.7.5) in the seven-point formula (4.7.6), you can't simply bisect intervals. But dividing into six subintervals works (there are six intervals between seven points).

To use the routine, you need to initialize an `Adapt` object with your required tolerance,

```
Adapt s(1.0e-6);
```

and then call the `integrate` function:

```
ans=s.integrate(func,a,b);
```

You should check that the desired tolerance could be met:

```
if (s.out_of_tolerance)
    cout << "Required tolerance may not be met" << endl;
```

The smallest allowed tolerance is 10 times the machine precision. If you enter a smaller tolerance, it gets reset internally. (The routine will work using the machine precision itself, but then it usually just takes lots of function evaluations for little additional benefit.)

The implementation of the `Adapt` object is given in a Webnote [2].

Adaptive quadrature is no panacea. The above routine has no special machinery to deal with singularities other than to refine the neighboring intervals. By using